

Sorting algorithms & Divide and Conquer

- Elementary Sorting Algorithms
- Merge Sort
- Divide and Conquer

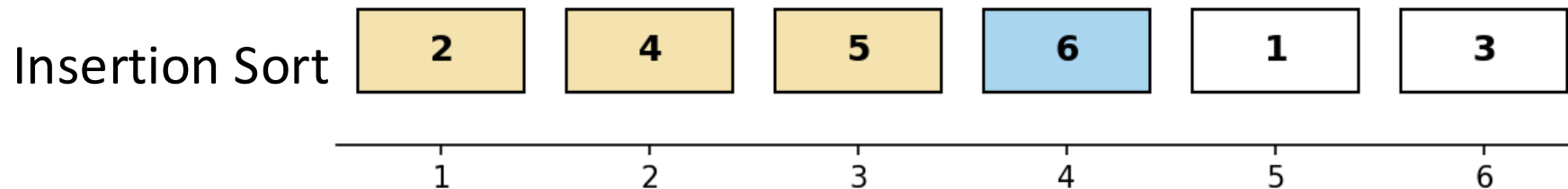
Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

Elementary Sorting Algorithms (Recap.)

The Problem of Sorting

- Input
 - A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- Output
 - Permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$
- Example
 - Input: 8 2 4 9 3 6
 - Output: 2 3 4 6 8 9
- Sorting is a fundamental operation:
 - Searching (Binary Search requires sorted arrays)
 - Data Processing (Efficient indexing in databases)
 - Graph Algorithms (Kruskal's algorithm for MST)
 - Bioinformatics (Sorting datasets before analysis)

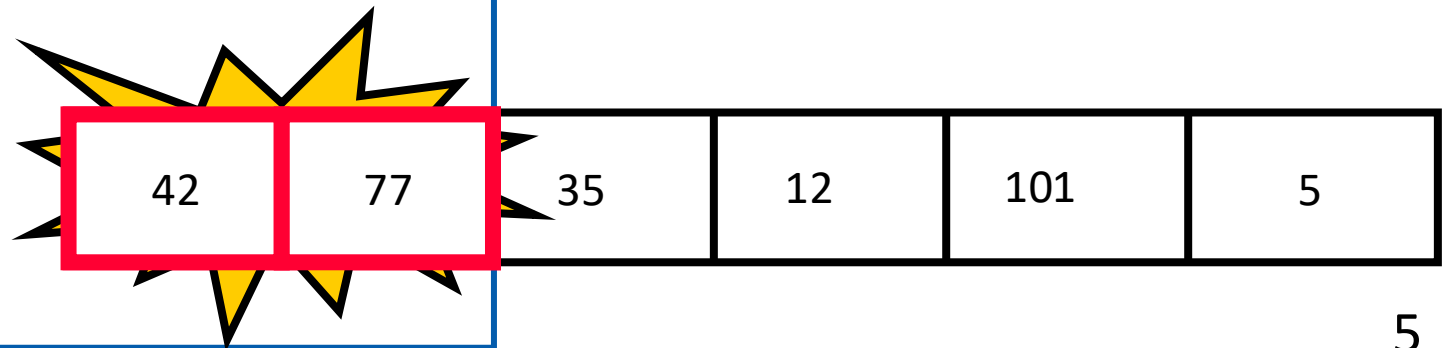
- Always consider the array has **two portions**: the **sorted portion** followed by the **unsorted portion**.
 - Initially, the sorted portion is empty, and the entire array is unsorted.
 - Enlarge the sorted portion by adding one more number in every iteration, and shrink the unsorted portion accordingly.



Bubble Sort (Recap.)

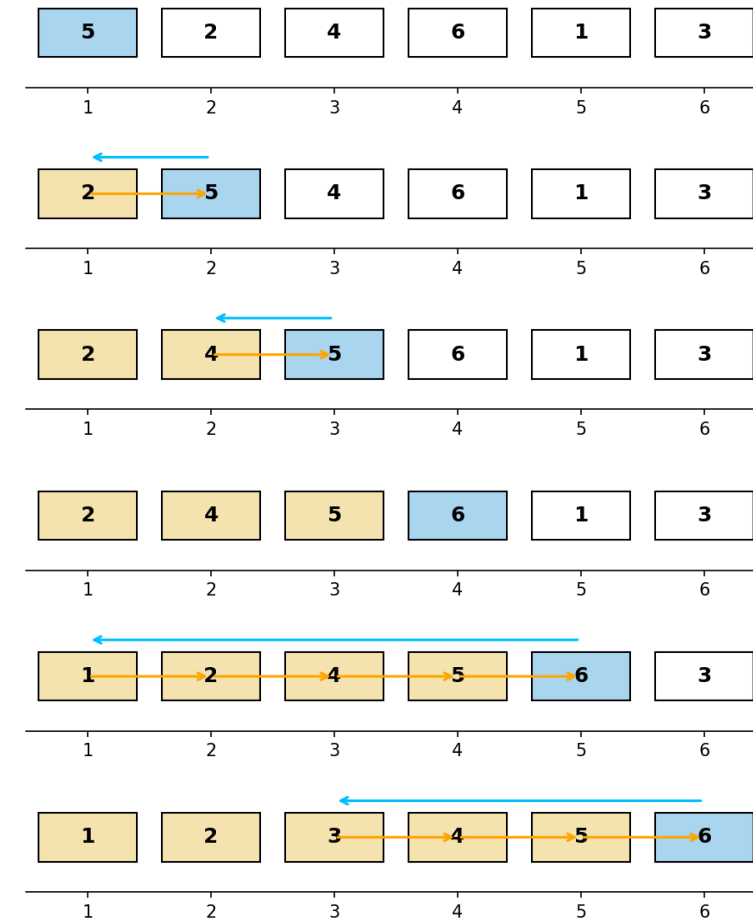
```
algorithm Bubblesort(A)
  to_do: index isoftype Num
  did_swap: Boolean
  to_do  $\leftarrow$  N - 1
  did_swap  $\leftarrow$  true

  while (to_do > 0) or (not did_swap):
    index  $\leftarrow$  1
    did_swap  $\leftarrow$  false
    while index < to_do + 1
      if(A[index] > A[index + 1]) then
        Swap(A[index], A[index + 1])
        did_swap  $\leftarrow$  true
      endif
      index  $\leftarrow$  index + 1
    end while
    to_do  $\leftarrow$  to_do - 1
  end while
```



Insertion Sort (Recap.)

Insertion Sort Step-by-Step



Algorithm InsertionSort(A)

Input: An array A with n elements

Output: A sorted in non-decreasing order

```
1. for i ← 2 to length(A) do
2.     key ← A[i] // Current element to be inserted
3.     j ← i - 1 // Start comparing from the previous element
4.     while j > 0 and A[j] > key do
5.         A[j + 1] ← A[j] // Shift element to the right
6.         j ← j - 1
7.     end while
8.     A[j + 1] ← key // Place key in the correct position
9. end for
10. return A // Sorted array
```

Selection sort (Recap.)

- Continuously finds the smallest element from the unsorted part and swaps it with the first unsorted position.

Input: An array A of size n (1-based index)
Output: A sorted array A in non-decreasing order
Algorithm SelectionSort(A, n):
 for i ← 1 to n do
 min_index ← i
 for j ← i+1 to n do
 if A[j] < A[min_index] then
 min_index ← j
 swap A[i] and A[min_index]
 return A

Pass	Unsorted Part	Min Element	Swap	Updated Array
1	[6, 3, 8, 5, 2]	2 (at index 5)	Swap A[1] ↔ A[5]	[2, 3, 8, 5, 6]
2	[3, 8, 5, 6]	3 (at index 2)	No swap needed	[2, 3, 8, 5, 6]
3	[8, 5, 6]	5 (at index 4)	Swap A[3] ↔ A[4]	[2, 3, 5, 8, 6]
4	[8, 6]	6 (at index 5)	Swap A[4] ↔ A[5]	[2, 3, 5, 6, 8]
5	[8]	No need to swap	Done	[2, 3, 5, 6, 8]

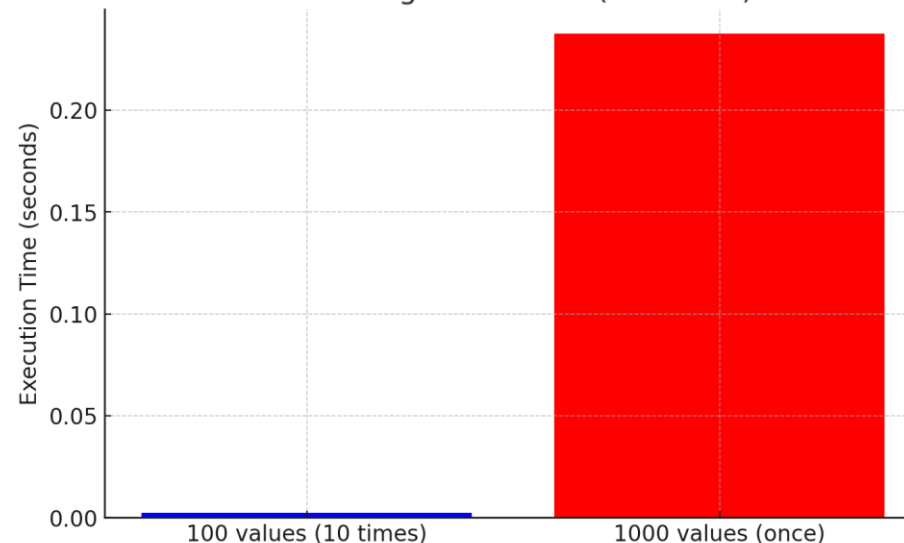
Merge Sort

The Problem with $O(n^2)$ Sorting Algorithms

Algorithm	Comparisons (Worst Case)	Swaps (Worst Case)
Bubble Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n)$
Insertion Sort	$O(n^2)$	$O(n)$

- We need a sorting method that **reduces swaps and comparisons**.

Bubble Sort Performance: Sorting 100 Values (10 Times) vs. 1000 Values (Once)



How Would You Sort 1000 Pieces of Paper?

- **Method A:** Uses Insertion Sort strategy (picking one paper at a time and inserting it in order).
- **Method B:** Uses Merge-like strategy (first sorts small sections, then merges them together).
 - How many comparisons are needed for merging two sorted sections?
- **Result:** Method B finishes much faster!
- **Lesson:** Sorting small sections first, then merging, is faster than moving elements one by one.
 - Fewer swaps and comparisons = faster sorting!
 - Merging sorted sections is easier than sorting everything from scratch.

Algorithm Merge-Sort $A[1 \dots n]$

1. If $n = 1$, done
2. Recursively sort $A[1 \dots \lceil n/2 \rceil]$
and $A[\lceil n/2 \rceil + 1 \dots n]$
3. “*Merge*” the 2 sorted lists

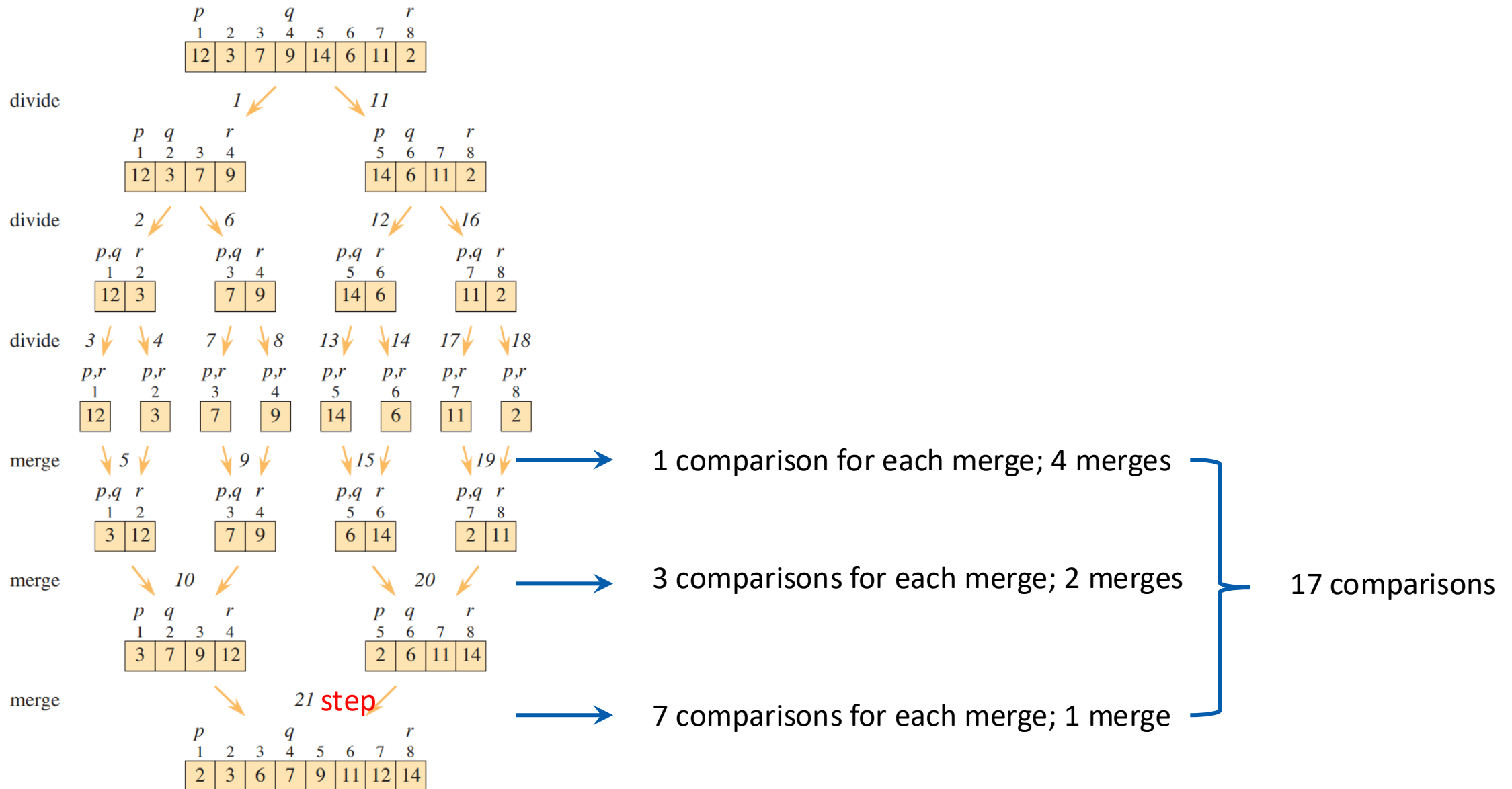
Key subroutine: **MERGE**

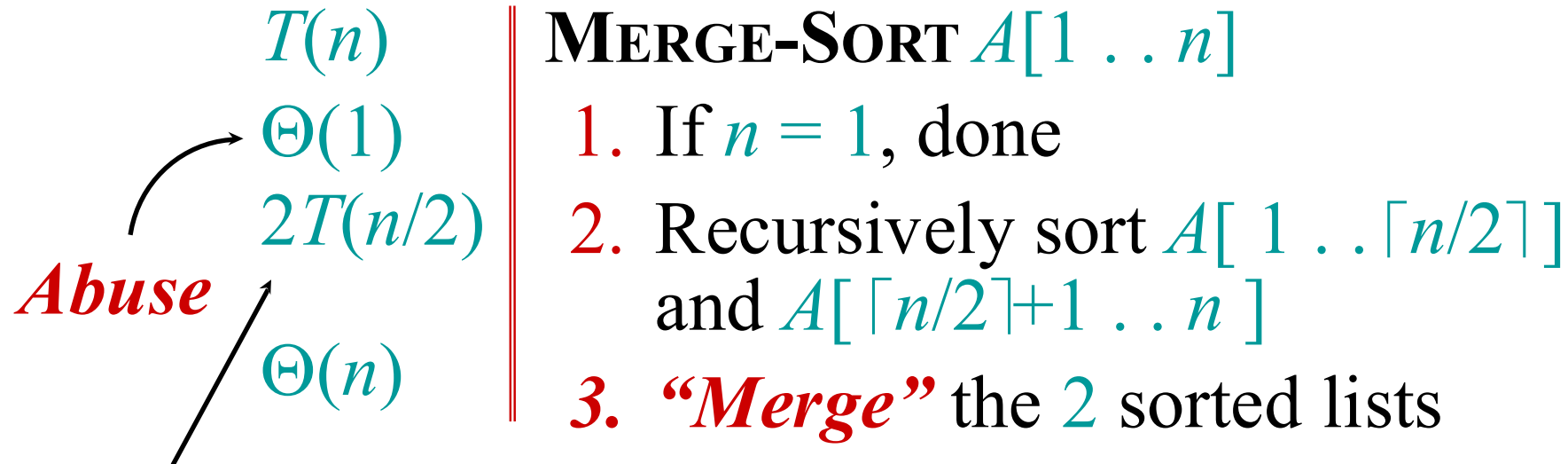
Merging Two Sorted Arrays (Algo. Merge)

20	12
13	11
7	9
2	1

Time = $\Theta(n)$ to merge a total
of n elements (linear time)

Merge-Sort Example





Sloppiness: Should be $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$, but it turns out not to matter asymptotically.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

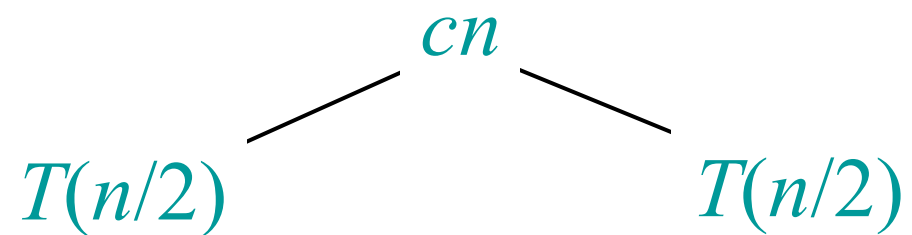
- We shall usually omit stating the base case when $T(n) = \Theta(1)$ for sufficiently small n , but only when it has no effect on the asymptotic solution to the recurrence.
- CLRS provides several ways to find a good upper bound on $T(n)$.

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

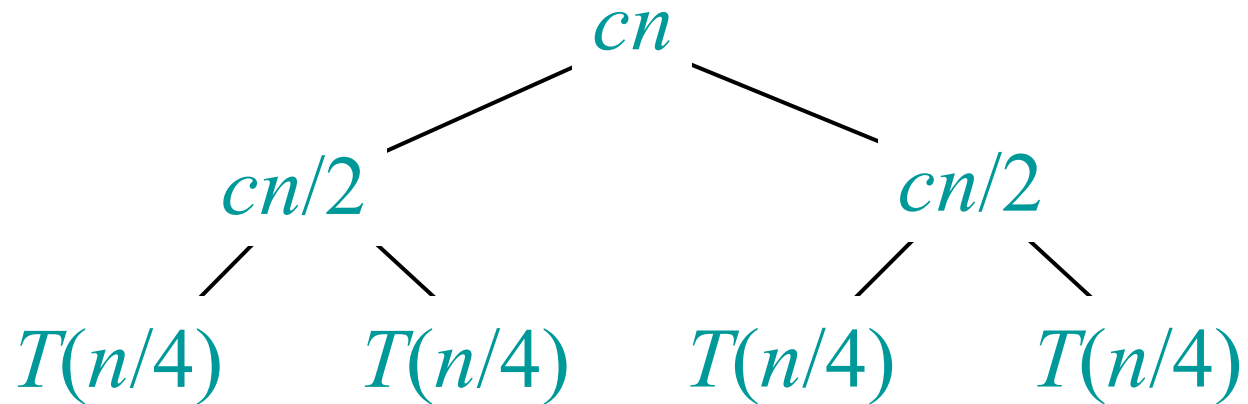
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$$T(n)$$

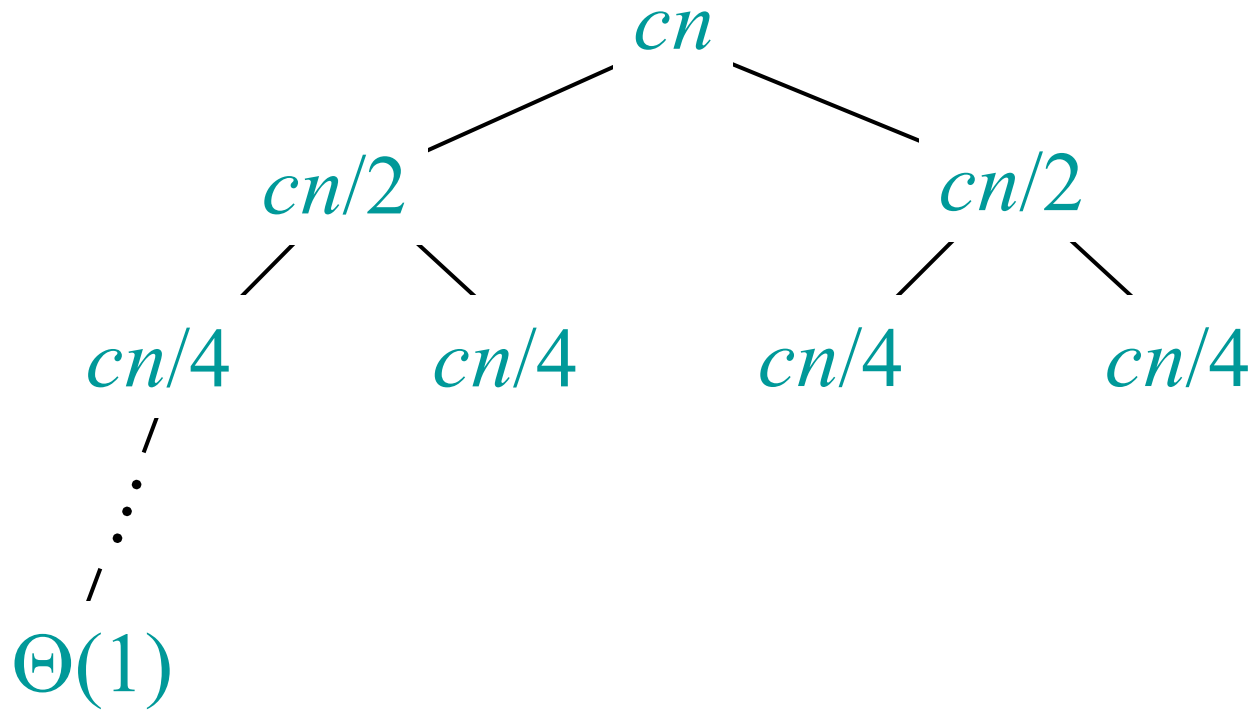
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



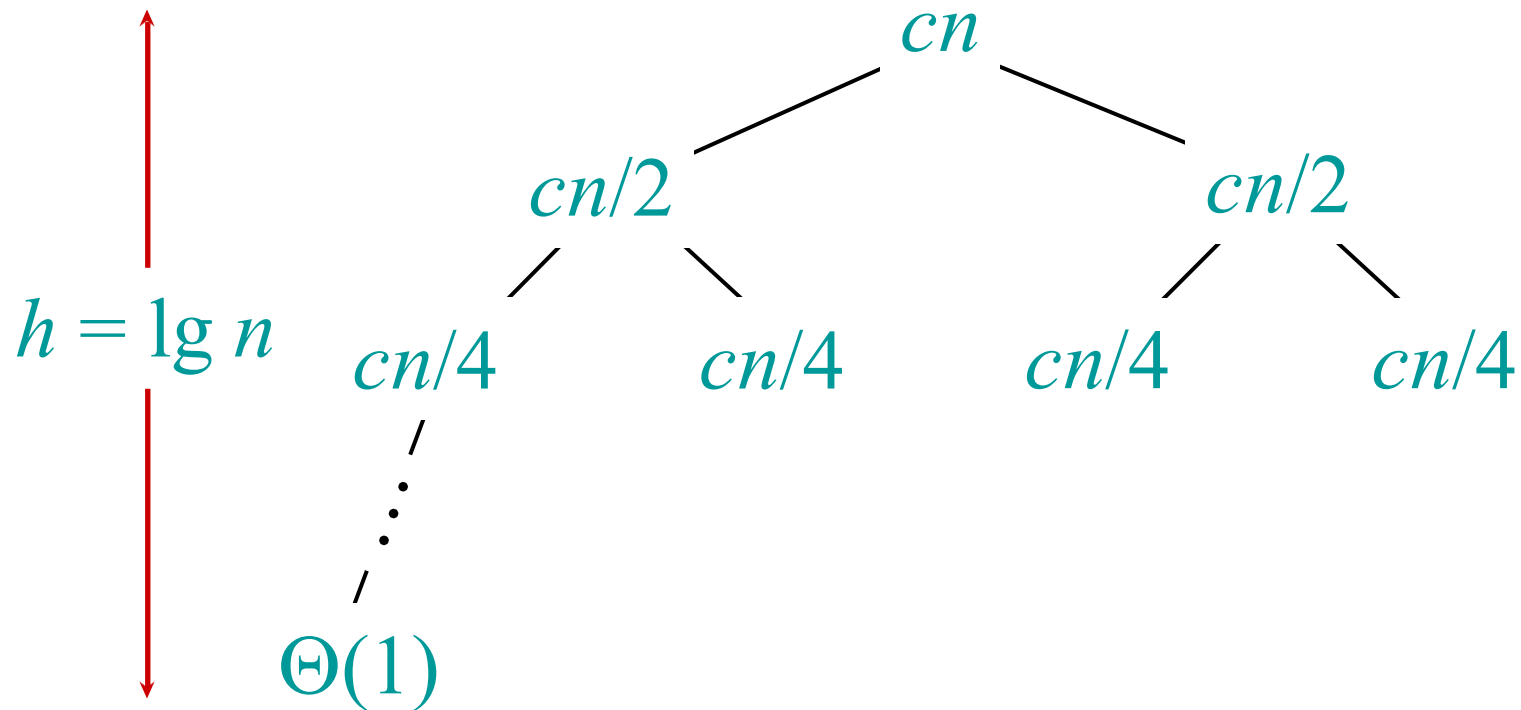
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



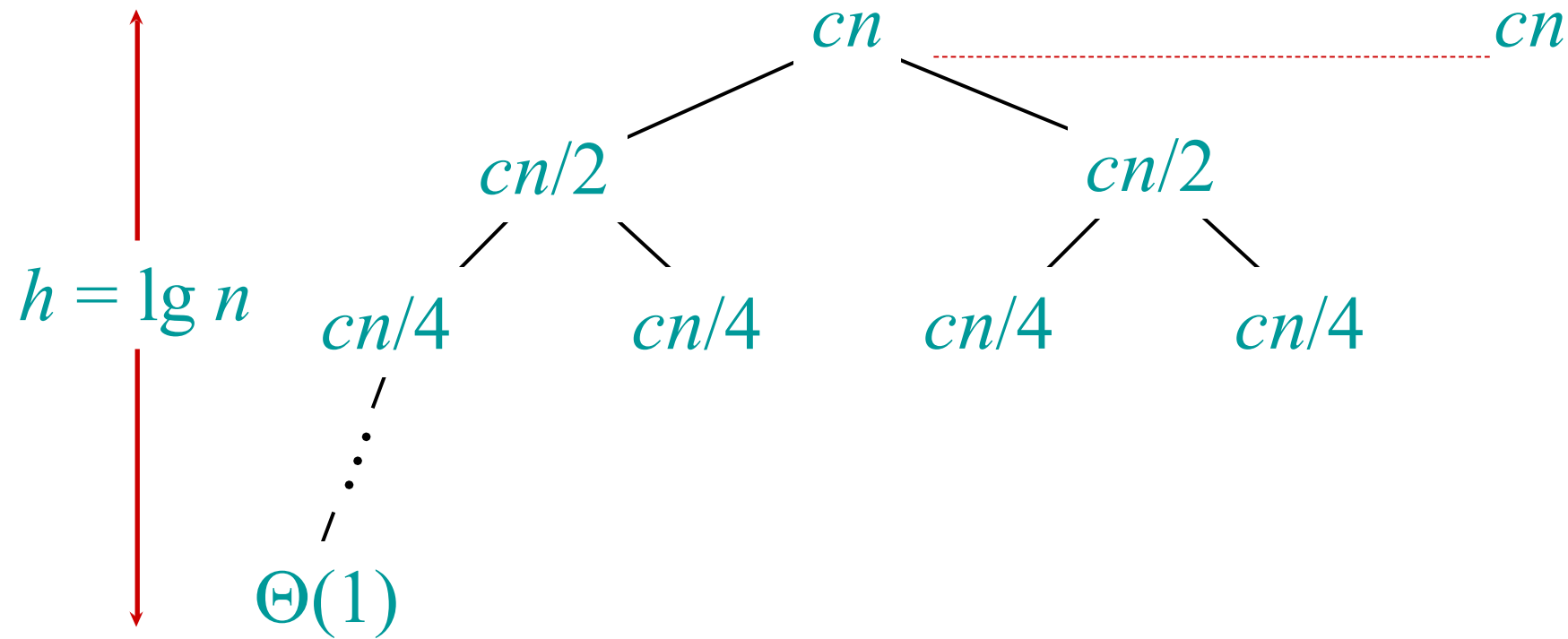
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

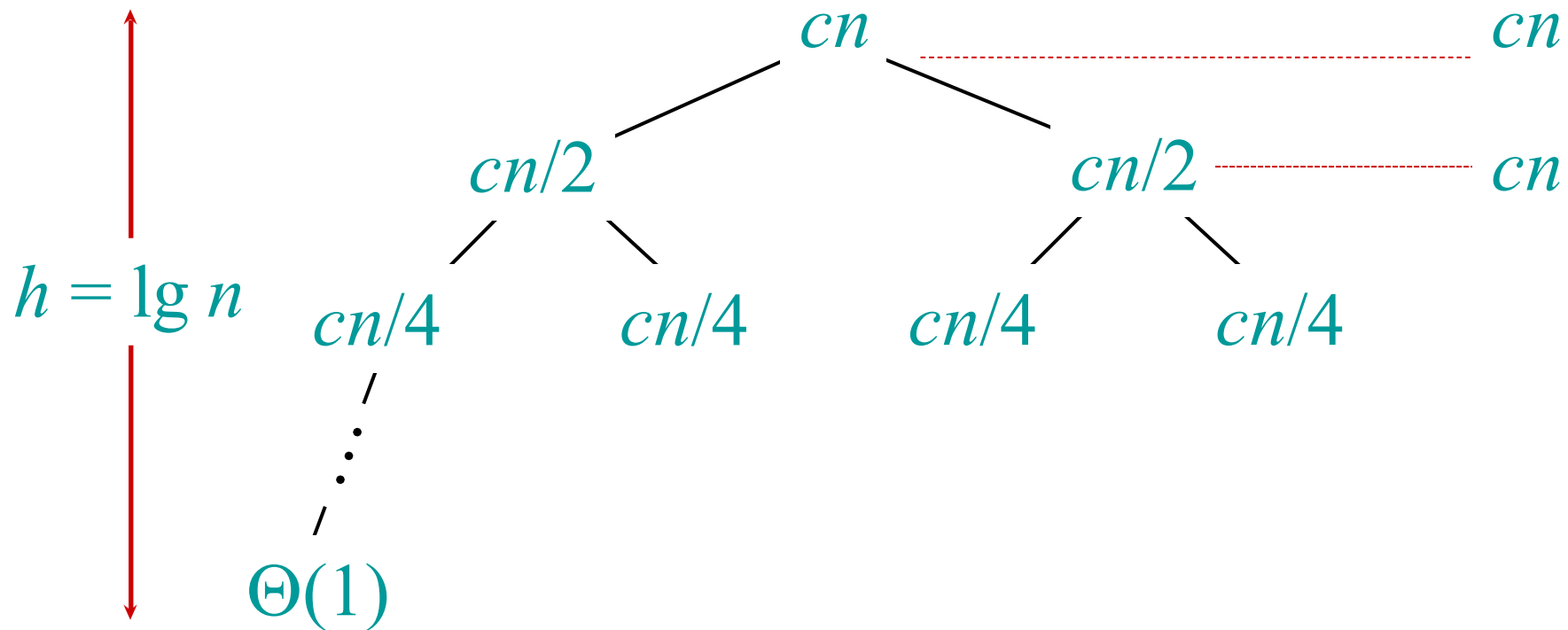


Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

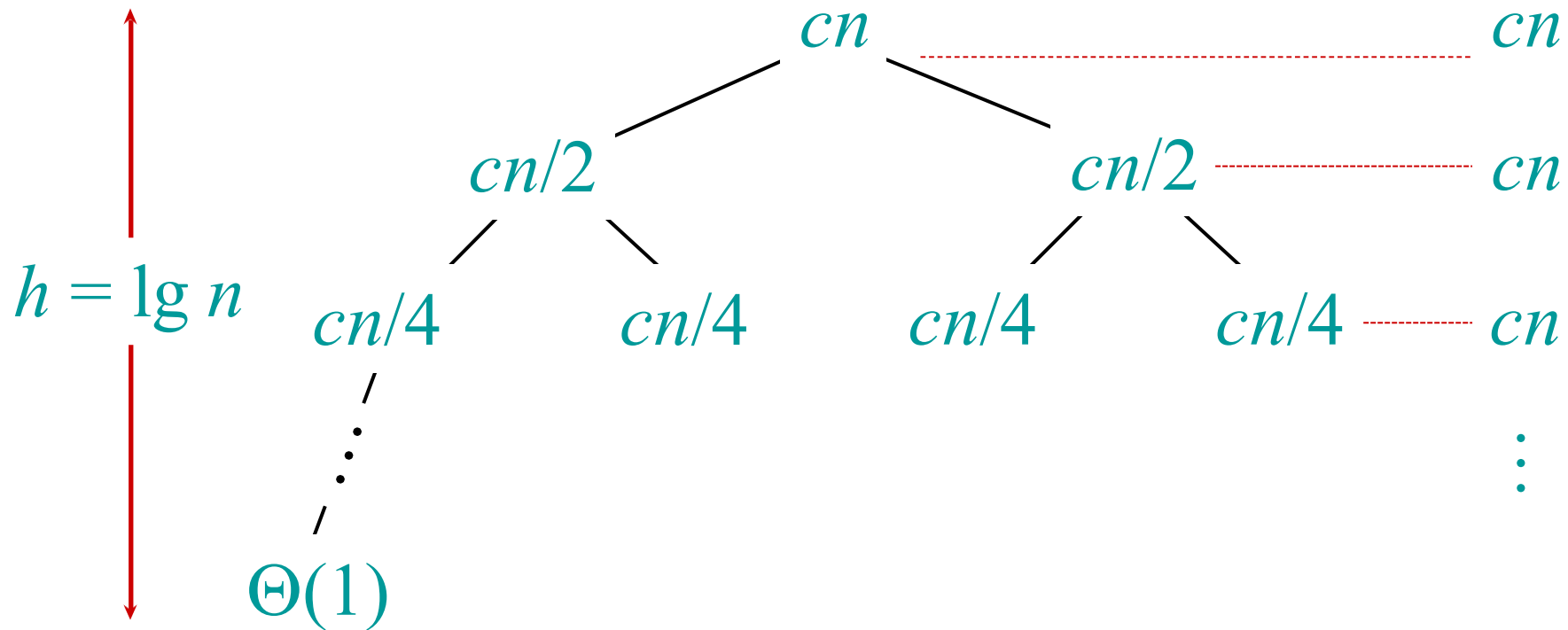


Recursion Tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

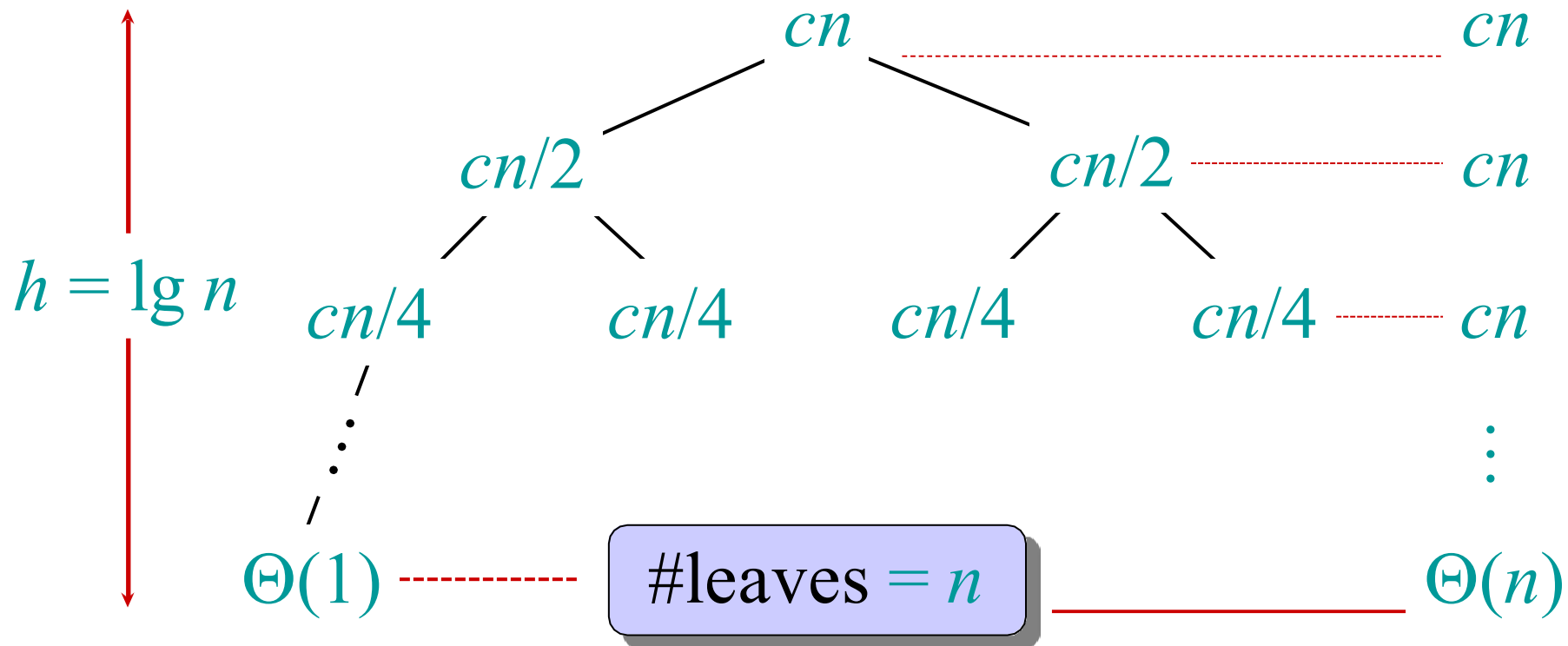


Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



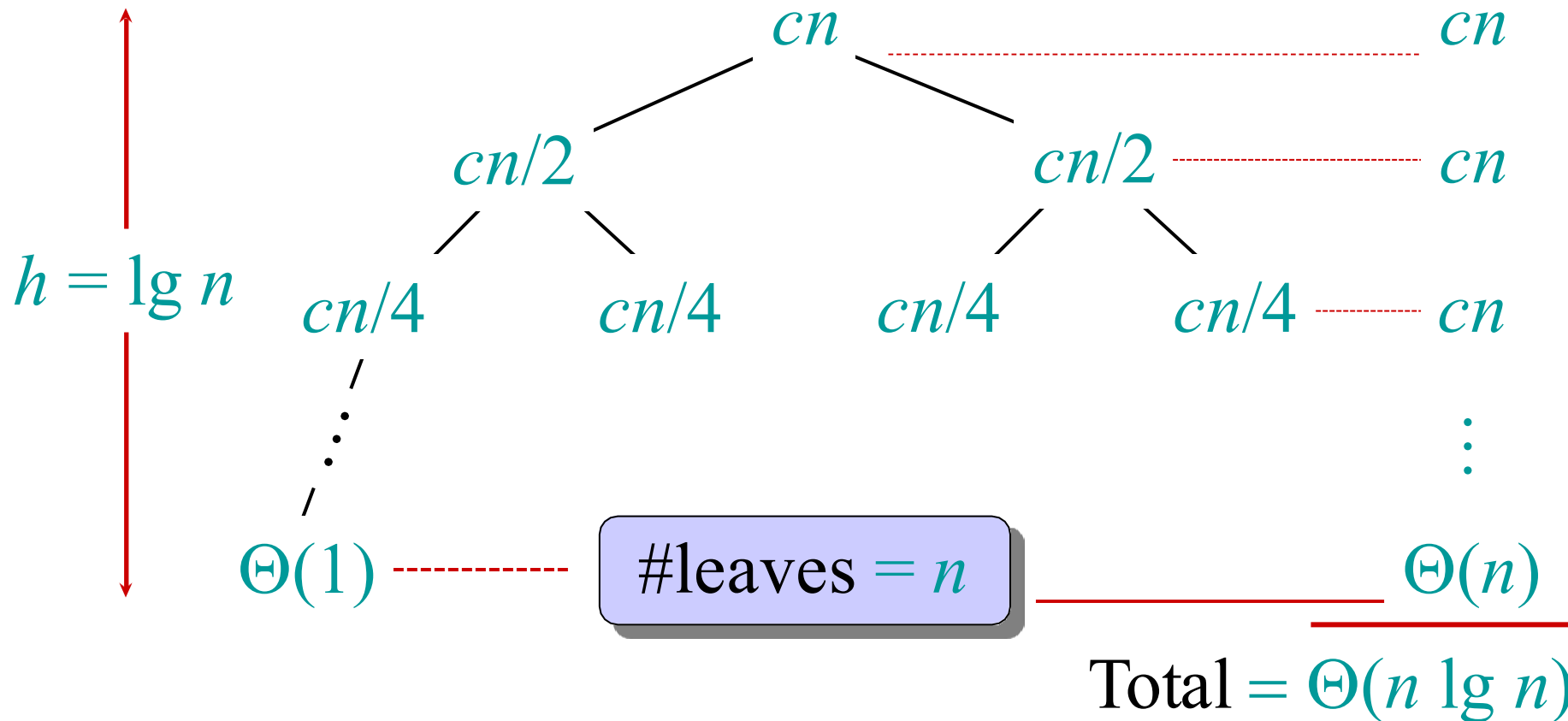
Recursion Tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Recursion Tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



- $\Theta(n \lg n)$ grows more slowly than $\Theta(n^2)$.
- Therefore, merge sort asymptotically beats insertion sort in the worst case.
- In practice, merge sort beats insertion sort for $n > 30$ or so.
- Go test it out for yourself!

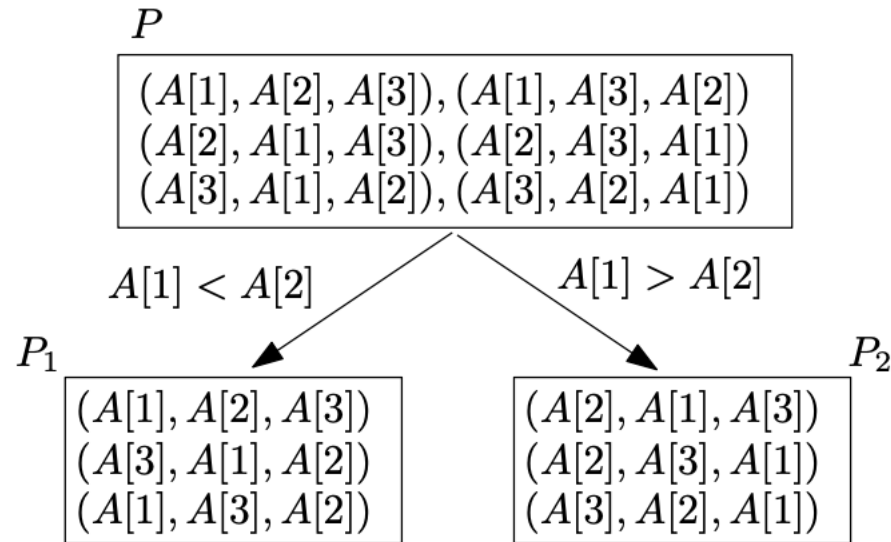
- Next, we will prove that Merge-Sort is optimal for **comparison-based algorithms**. In other words, every such algorithm must incur $\Omega(n \log n)$ time on at least one input.
- There are **$n!$** different ways to permute the n elements in the input array **A** .
- Example: For $n = 3$, 6 permutations:
 $A[1], A[2], A[3]$
 $A[1], A[3], A[2]$
 $A[2], A[1], A[3]$
 $A[2], A[3], A[1]$
 $A[3], A[1], A[2]$
 $A[3], A[2], A[1]$
- The goal of sorting is essentially to decide which of $n!$ Permutations is the final sorted order.

Formally, such an algorithm works by continuously shrinking a pool P of possible permutations.

- At the beginning, P contains all the $n!$ permutations.
- Every comparison allows the algorithm to discard all those permutations in P that are inconsistent with the comparison's result.
- Eventually, P has only 1 permutation left, which is thus the final sorted order.

In other words, at any moment, all the permutations that remain in P are possible results. The algorithm cannot terminate as long as $|P| \geq 2$.

- Shrinking the Pool: An Example



- In general, each comparison allows us to shrink P to either P_1 or P_2 .

Framework

1. $P \leftarrow$ all the $n!$ permutations of A
2. **while** $|P| > 1$
3. make a comparison between elements e_1 and e_2
4. **if** $e_1 < e_2$ **then**
5. $P \leftarrow P_1$, where P_1 is the set of permutations in P
 consistent with $e_1 < e_2$
6. **else**
7. $P \leftarrow P_2$, where P_2 is the set of permutations in P
 consistent with $e_1 > e_2$
8. **return** the permutation in P

Various algorithms differ in how they implement Step 3.

A Worst-Case Lower Bound

Formally, such an algorithm works by continuously shrinking a pool P of possible permutations.

- Note that one of P_1 and P_2 contains at least half of the permutations in P (i.e., either $|P_1| \geq |P|/2$ or $|P_2| \geq |P|/2$).
- The worst case happens when P always shrinks to the **larger** set between P_1 and P_2 .
- In this case, the size of P shrinks by at most half after each comparison.
- Hence, the number of comparisons required before $|P|$ decreases to 1 is $\log_2(n!)$.

The next slide shows $\log_2(n!) = \Omega(n \log n)$.

A Worst-Case Lower Bound

$$\begin{aligned}\log_2(n!) &= \sum_{i=1}^n \log_2 i \\ &\geq \sum_{i=n/2}^n \log_2 i \\ &\geq (n/2) \log_2(n/2) \\ &= \Omega(n \log n)\end{aligned}$$

We now conclude that any comparison-based algorithm must incur $\Omega(n \log n)$ time sorting n elements in the worst case.

Divide-and-Conquer

The Divide-and-Conquer Design Paradigm

1. *Divide* the problem (instance) into subproblems.
2. *Conquer* the subproblems by solving them recursively.
3. *Combine* subproblem solutions.

- 1. Divide:* Trivial.
- 2. Conquer:* Recursively sort 2 subarrays.
- 3. Combine:* Linear-time merge.

1. **Divide:** Trivial.
2. **Conquer:** Recursively sort 2 subarrays.
3. **Combine:** Linear-time merge.

$$T(n) = 2T(n/2) + \Theta(n)$$

subproblems

subproblem size

work dividing and combining

Master Theorem (Reprise)

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a})$$

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^l n)$, constant $l \geq 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{l+1} n)$$

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$,
and regularity condition $a f(n/b) \leq c f(n)$,
constant $c < 1$ for all sufficiently large n

$$\Rightarrow T(n) = \Theta(f(n))$$

Master Theorem (Reprise)

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \varepsilon})$, constant $\varepsilon > 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a})$$

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^l n)$, constant $l \geq 0$

$$\Rightarrow T(n) = \Theta(n^{\log_b a} \lg^{l+1} n)$$

CASE 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$, constant $\varepsilon > 0$,
and regularity condition $a f(n/b) \leq c f(n)$,
constant $c < 1$ for all sufficiently large n

$$\Rightarrow T(n) = \Theta(f(n))$$

Merge sort: $a = 2, b = 2 \Rightarrow n^{\log_b a} = n^{\log_2 2} = n$

$$\Rightarrow \text{CASE 2 } (l = 0) \Rightarrow T(n) = \Theta(n \lg n)$$

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search 1 subarray.
- 3. *Combine:*** Trivial.

Master Theorem (Proof)

$$T(n) = a T(n/b) + f(n)$$

Try to solve case 2 at home!

$$T(n) = \Theta(n^{\log_b a}) + \sum_{i=0}^{k-1} a^i f\left(\frac{n}{b^i}\right), \longrightarrow g(n) = \sum_{i=0}^{k-1} a^i f\left(\frac{n}{b^i}\right)$$

$$n = b^k, k = \log_b n, a^k = a^{\log_b n} = n^{\log_b a}$$

For case1: $f(n) = O(n^{(\log_b a) - \epsilon}), \epsilon > 0$

$$\begin{aligned} \text{We have: } g(n) &= O\left(\sum_{i=0}^{k-1} a^i \left(\frac{n}{b^i}\right)^{(\log_b a) - \epsilon}\right) \\ &= O\left(n^{(\log_b a) - \epsilon} \sum_{i=0}^{k-1} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^i\right) \\ &= O\left(n^{(\log_b a) - \epsilon} \sum_{i=0}^{k-1} (b^\epsilon)^i\right) \\ &= O\left(n^{(\log_b a) - \epsilon} \sum_{i=0}^{k-1} (b^\epsilon)^i\right) \\ &= O\left(n^{(\log_b a) - \epsilon} \frac{n^\epsilon - 1}{b^\epsilon - 1}\right) \\ &= O(n^{\log_b a}) \end{aligned}$$

For case3: $f(n) = \Omega(n^{(\log_b a) + \epsilon}), \epsilon > 0$

$$af\left(\frac{n}{b}\right) \leq cf(n), c < 1$$

We have: $af\left(\frac{n}{b^2}\right) \leq cf\left(\frac{n}{b}\right)$

$\vdots$

$$af\left(\frac{n}{b^i}\right) \leq cf\left(\frac{n}{b^{i-1}}\right)$$

Multiply both sides: $a^i f\left(\frac{n}{b^i}\right) \leq c^i f(n)$

$$\begin{aligned} g(n) &= \sum_{i=0}^{k-1} a^i f\left(\frac{n}{b^i}\right) \leq \sum_{i=0}^{k-1} c^i f(n) = f(n) \sum_{i=0}^{k-1} c^i \\ &\leq f(n) \frac{1}{1-c} = \Theta(f(n)) \end{aligned}$$

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search 1 subarray.
- 3. *Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search **1** subarray.
- 3. Combine:** Trivial.

Example: Find **9**



Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search 1 subarray.
- 3. Combine:** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search 1 subarray.
- 3. Combine:** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search 1 subarray.
- 3. *Combine:*** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search 1 subarray.
- 3. Combine:** Trivial.

Example: Find 9

3 5 7 8 9 12 15

Recurrence for Binary Search

$$T(n) = 1 T(n/2) + \Theta(1)$$

subproblems *subproblem size* *work dividing and combining*

Recurrence for Binary Search

$$T(n) = 1 T(n/2) + \Theta(1)$$

subproblems *subproblem size* *work dividing and combining*

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow \text{CASE 2 } (l=0) \\ \Rightarrow T(n) = \Theta(\lg n).$$

Powering a Number

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

Problem: Compute a^n , where $n \in \mathbf{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = \Theta(\lg n).$$

Recursive definition:

$$F_n = \begin{cases} 1 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 ...

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 ...

Naive recursive algorithm: $\Omega(\phi^n)$

(exponential time), where $\phi = (1 + \sqrt{5})/2$
is the *golden ratio*.

Bottom-up:

- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.

Bottom-up:

- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.

Naive recursive squaring:

$F_n = \phi^n / 5$ rounded to the nearest integer.

- Recursive squaring: $\Theta(\lg n)$ time.
- This method is unreliable, since floating-point arithmetic is prone to round-off errors.

Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

Algorithm: Recursive squaring.

Time = $\Theta(\lg n)$.

Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

Algorithm: Recursive squaring.

Time = $\Theta(\lg n)$.

Proof of theorem. (Induction on n .)

Base ($n = 1$):
$$\begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^1$$

Inductive step ($n \geq 2$):

$$\begin{aligned} \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} &= \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \end{aligned}$$

Or Equivalently

$$\begin{bmatrix} F_{n+1} & F_n \end{bmatrix} = \begin{bmatrix} F_n & F_{n-1} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Matrix Multiplication

Input: $A = [a_{ij}], B = [b_{ij}].$
Output: $C = [c_{ij}] = A \cdot B.$ $\left. \vphantom{\begin{matrix} A \\ B \\ C \end{matrix}} \right\} i, j = 1, 2, \dots, n.$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```

Running time = $\Theta(n^3)$

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$r = ae + bg$$

$$s = af + bh$$

$$t = ce + dg$$

$$u = cf + dh$$

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices

Divide-and-Conquer Algorithm

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$\begin{matrix} r = ae + bg \\ s = af + bh \\ t = ce + dh \\ u = cf + dg \end{matrix} \left\{ \begin{array}{l} \text{recursive} \\ 8 \text{ mults of } (n/2) \times (n/2) \text{ submatrices} \\ 4 \text{ adds of } (n/2) \times (n/2) \text{ submatrices} \end{array} \right.$$

Analysis of D&C Algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices *submatrix size* *work adding submatrices*

Analysis of D&C Algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices *submatrix size* *work adding submatrices*

$$n^{\log ba} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

Analysis of D&C Algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices *submatrix size* *work adding submatrices*

$$n^{\log ba} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.

- Multiply 2×2 matrices with only 7 recursive mults.

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.

Note: No reliance on commutativity of mult!

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$\begin{aligned} r &= P_5 + P_4 - P_2 + P_6 \\ &= (a + d)(e + h) \\ &\quad + d(g - e) - (a + b)h \\ &\quad + (b - d)(g + h) \\ &= ae + ah + de + dh \\ &\quad + dg - de - ah - bh \\ &\quad + bg + bh - dg - dh \\ &= ae + bg \end{aligned}$$

- 1. *Divide:*** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
- 2. *Conquer:*** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
- 3. *Combine:*** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.

- 1. *Divide*:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
- 2. *Conquer*:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
- 3. *Combine*:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log ba} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log ba} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log ba} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

Best to date (of theoretical interest only): $\Theta(n^{2.376...})$.

- Divide and conquer is just one of several powerful techniques for algorithm design.
- Divide-and-conquer algorithms can be analyzed using recurrences and the master method (so practice this math).
- The divide-and-conquer strategy often leads to efficient algorithms.